HDOC 16

NAME: DARSHIL RAJ DATE: 15/10/26
SAP ID: 590040196

Program to Print the Binary Representation of a Number

This document provides a clean, well-commented implementation in C that takes a non-negative integer as input and prints its equivalent binary representation. The program repeatedly divides the number by 2, stores each remainder, and then prints the remainders in reverse order.

1. C Implementation

Step 1: Understand the Logic

To convert a decimal number (e.g., 10) to binary, divide it by 2 repeatedly and note the remainder (n % 2) each time. Then reduce the number by integer division (n /= 2). The binary number is the sequence of remainders read from **last to first**.

- **Initialization:** count = 0 (number of binary digits stored).
- **Condition:** n > 0, so the loop continues while the number is not exhausted.
- **Update:** n /= 2 after each remainder is stored.
- **Special case:** if the input is 0, the binary representation is simply 0.

Step 2: Define Variables and Data Types

Use int for the input number (num), a working copy (n), a loop index (i) and a digit counter (count). Use an int array binary[32] to store the remainders (an int has at most 32 bits).

Step 3: Take User Input

Prompt the user using printf("Enter a non-negative integer: "); and read using scanf("%d", &num);. Negative input is rejected with a message.

Step 4: Convert Using a while Loop

1. Copy num into n so the original can be displayed.
2. Store binary[count++] = n % 2.
3. Update n = n / 2 until it reaches 0.
4. Print the array from index count-1 down to 0.

Complete C Code

```
/*
 * Q26 (User Inputs, Operations & Output)
 * Write a program to take a number as input and print its
 * equivalent binary representation.
 */
#include <stdio.h>

int main() {
    int num, n, i, count = 0;
    int binary[32];                 // Stores binary digits (remainders)

    // Step 1: Read the value of num
    printf("Enter a non-negative integer: ");
    scanf("%d", &num);

    if (num < 0) {
        printf("Please enter a non-negative number.\n");
        return 1;
    }

    n = num;                        // Save original for output display

    // Step 2: Special case - zero has binary representation 0
    if (n == 0) {
        binary[count++] = 0;
    }

    // Step 3: Repeatedly divide by 2 and store remainders
    while (n > 0) {
        binary[count++] = n % 2;
        n /= 2;
    }

    // Step 4: Print remainders in reverse order
    printf("Binary of %d is: ", num);
    for (i = count - 1; i >= 0; i--) {
        printf("%d", binary[i]);
    }
    printf("\n");

    return 0;
}
```

Program Output (Sample Runs)

```
Run 1:  $ gcc hdoc16.c -o hdoc16
        $ ./hdoc16
        Enter a non-negative integer: 10
        Binary of 10 is: 1010

Run 2:  $ ./hdoc16
        Enter a non-negative integer: 25
        Binary of 25 is: 11001

Run 3:  $ ./hdoc16
        Enter a non-negative integer: 255
        Binary of 255 is: 11111111

Run 4:  $ ./hdoc16
        Enter a non-negative integer: 0
        Binary of 0 is: 0

Run 5:  $ ./hdoc16
        Enter a non-negative integer: -5
        Please enter a non-negative number.
```

Dry Run with Test Cases

Sample Test Case 1

Input: num = 10
Calculation:
Start: n = 10, count = 0.
Pass 1: 10 > 0 is TRUE. remainder = 10 % 2 = 0, binary[0] = 0, n = 5.
Pass 2: 5 > 0 is TRUE. remainder = 1, binary[1] = 1, n = 2.
Pass 3: 2 > 0 is TRUE. remainder = 0, binary[2] = 0, n = 1.
Pass 4: 1 > 0 is TRUE. remainder = 1, binary[3] = 1, n = 0.
Check: 0 > 0 is FALSE, loop terminates. Array = [0, 1, 0, 1].
Printing in reverse: 1 0 1 0
Output: Binary of 10 is: 1010

Sample Test Case 2

Input: num = 5
Calculation:
Pass 1: remainder = 1, n = 2.
Pass 2: remainder = 0, n = 1.
Pass 3: remainder = 1, n = 0. Array = [1, 0, 1].
Output: Binary of 5 is: 101

Sample Test Case 3

Input: num = 0
Calculation: The special case n == 0 stores a single 0. The while loop does not execute (0 > 0 is FALSE).
Output: Binary of 0 is: 0